

2025 牛客 暑期多校训练营

南京大学命题组



F - For the Treasury!

题目大意

- 给定一个长度为 n 的数组 $a_1, a_2, \dots, a_n$ ，一个正整数 $1 \leq k \leq n$ 以及一个代表代价的数字 c 。可以进行任意次交换相邻两个数的操作，求

$$\sum_{i=1}^k a_i - \text{num} \cdot c$$

的最大值，其中 num 表示执行交换操作的次数。

- $1 \leq n \leq 3 \times 10^5, 0 \leq c \leq 10^9$ 。

F - For the Treasury!

题解

- 考虑希望最终数组前 k 个元素为初始数组中子序列 $a_{x_1}, a_{x_2}, \dots, a_{x_k}$ 时所需的最小交换次数。显然可以假设 $1 \leq x_1 < x_2 < \dots < x_k \leq n$ 并在交换过程中保证这些数字相对位置不变。
- 容易发现所需交换次数为

$$\sum_{i=1}^k (x_i - i) = \sum_{i=1}^k x_i - \sum_{i=1}^k i.$$

- 因此将所有数字按照 $a_i - ci$ 排序取前 k 大后再减去 $\frac{k(k+1)}{2} \cdot c$ 即可得到答案。时间复杂度为 $O(n \log n)$ 。

B - Blind Alley

题目大意

- 给一个大小为 $N \times M$ 的含有障碍物的地图，判断是否有可能因视野问题走入死胡同。
- $NM \leq 10^6$ 。

B - Blind Alley

题解

- 本题可能存在若干种做法，我们这里提供一种比较自然的做法。
- 第一步：找出所有可以从起点到达但不能到达终点的格子（踏入这些格子即必定走入死胡同）。这些格子可以通过正向 BFS 以及反向 BFS 找出；
- 第二步：判定这些格子是否足够“深”，以至于误导玩家踏入这些格子。这等价于问是否存在某个格子，可以通过这些“死胡同”格子可以到达其后的 K 列。这一步可以通过由后往前的动态规划实现。
- 整体时间复杂度： $O(NM \log NM)$ 或 $O(NM)$ 。

G - Ghost in the Parentheses

题目大意

- 给一个合法括号序列 s ，每个字符以 $1/2$ 的概率变成问号字符。问能唯一还原合法括号序列的概率。
- $|s| \leq 10^6$ 。

G - Ghost in the Parentheses

题解

- 将 '(' 替换为 $+1$, ')' 替换为 -1 。
- 子序列满足条件当且仅当
 - 1 子序列为若干个 $+1$ 与若干个 -1 的拼接;
 - 2 若该子序列既有 $+1$, 又有 -1 , 令 i 为子序列中最后一个 $+1$ 在原序列中对应的位置, j 为子序列中第一个 -1 在原序列中对应的位置, 则交换原序列中第 i 号位置与第 j 号位置, 序列的前缀和不再始终非负。

G-Ghost in the Parentheses

题解

- 第二个条件等价于 $\min_{i \leq k < j} \text{prefix_sum}_k \leq 1$ 。
- 利用这些条件，我们只需要枚举 i 的位置，找到最小的 j 满足上述条件，即可在 $O(1)$ 时间内计数。总的时间复杂度是 $O(|s|)$ 或 $O(|s| \log |s|)$ 。

I - I, Box

题目大意

- 给定 $n \times m$ 的推箱子地图，但是规则变更为不需要人去推，而是箱子可以自己移动（同样不能移动到障碍物或者其他箱子上）。要求判断是否存在解，如果存在解要求构造出一组总移动次数不超过 10^5 的解。
- $1 \leq n, m \leq 50$ 。输入保证地图上存在至多 50 个箱子。

I - I, Box

题解

- 考虑整个地图被障碍物分成的多个连通区域。如果某个连通区域中箱子数量不等于终点数量，那么显然无解。
- 否则原问题一定存在解。每次考虑通过选取一条不经过障碍物的最短路径，将一个不在终点上的箱子推到一个上面没有箱子的终点上。如果这条路径上依次存在其他箱子 $a_1, a_2, \dots, a_\ell$ ，只需要先将 a_ℓ 推到终点，将 $a_{\ell-1}$ 推到 a_ℓ 原先所在位置... 并以此类推即可。
- 由于每次选取的是最短路，可以证明这样的操作次数一定不超过 10^5 。
总的时间复杂度为 $O(50nm)$ 。



D - Determinant of 01-Matirx

题目大意

- 构造一个 01 矩阵，其行列式为给定值 N 。
- $1 \leq N \leq 10^9$ 。

D - Determinant of 01-Matrix

题解

- 本题有多种构造方式，我们提供一种可行思路。
- 首先考虑如何用 $0, \pm 1, 2$ 构造一个满足条件的矩阵。

D - Determinant of 01-Matrix

题解

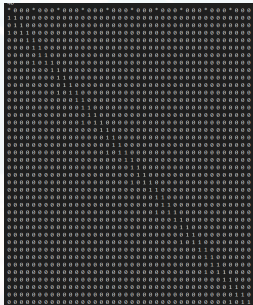
- 考虑如下矩阵，该矩阵去掉第一行第 K 列后行列式为 2^{k-1} ，因此我们可以利用 N 的二进制划分，将 $*$ 填入 -1 到 1 之间的数字，构造出行列式恰为 N 的矩阵。

$$A = \begin{bmatrix} * & * & * & \dots & * & * \\ 2 & 1 & 0 & \dots & 0 & 0 \\ 0 & 2 & 1 & \dots & 0 & 0 \\ & & & \dots & & \\ 0 & 0 & 0 & \dots & 2 & 1 \end{bmatrix}$$

D - Determinant of 01-Matrix

题解

- 如只用 0 和 1，我们考虑利用行列式为 2 的矩阵替换数字 2。以下为一种可行解，其中 * 可以填入 0 或 1。



L - Ladder Challenge

题目大意

- 有 N 个选手，分数分别为 $a_1 < a_2 < \dots < a_n$ 。
- 小明作为参赛选手，每次在所有分数比他高的人中选择分数最低的人，然后他获得 1 分，对方扣除 1 分。
- 有 Q 次询问，每次询问如果小明当前分数是 x ，多少次操作后他可以到达排名 y 。
- $N, Q \leq 2 \cdot 10^5, x, a_i \leq 10^9$ 。

L - Ladder Challenge

题解

- 对于每个询问，需要知道小明在到达排名 y 时的分数。
- 相当于回答对于一个区间 $[L, R]$ ，初始值为 x ，对于 $i = L, L + 1, \dots, R$ 依次执行：

$$x \leftarrow \left\lceil \frac{x + a_i}{2} \right\rceil$$

- 我们记 $f(L, R, x)$ 为上述操作完后得到的 x 的值。
- 可以发现， $f(L, R, x)$ 总可以写成以下形式：

$$f(L, R, x) = \left\lceil \frac{x + C}{2^{R-L+1}} \right\rceil$$

Ladder Challenge

L - Ladder Challenge

题解

- 区间 $[L, R]$ 的长度越长，最终的答案与 x 的初值关系“越小”。
- 由于 $x, a_i \leq A = 10^9$ ，可以发现，当区间长度 $\geq \log A = 30$ 时，对于 $f(L, R, 0)$ 与 $f(L, R, A)$ 至多差 1。
- 对于每个长度为 $B = 30$ 的区间，求出 $f(L, R, 0)$ 以及最小的正整数 x 满足 $f(L, R, x) = f(L, R, 0) + 1$ 。
- 考虑倍增，对于每个长度为 $30 \cdot 2^k$ 的区间，做出同样的预处理。
- 这样对于每个询问，可以在 $O(\log A + \log n)$ 时间内回答。
- 复杂度： $O((n + q)(\log n + \log A))$ 。

E - Echoes of 24

题目大意

- 一棵树，点有点权。每次询问一条路径，初始值为出发点的权值，然后依次加或乘上路径上的数，问你最终能不能恰好得到 24。
- $n, q \leq 5 \times 10^5, 1 \leq a_i \leq 10^9$ ，带修。

E - Echoes of 24

- 考虑在数组上最基础的做法。令 dp_i 表示当前能否达到数字 i ，那么假设当前要考虑的数字为 x ，那么由 dp_i 可以转移到 dp'_{i+x} 和 $dp'_{i \times x}$ 。
- 我们需要注意到，如果在这个区间中出现了超过 12 个大于 1 的数字，那么它一定是不合法的。因为他们最少相加起来也会超过 $12 \times 2 = 24$ ，无法再减小。
- 同时，相邻两个大于 1 的数之间，最多只有 24 个 1 是有用的。因为用到最多的 1 的时候也只可能是 24 个 1 相加得到 24，再多的 1 只能用来相乘。

E - Echoes of 24

- 考虑放在树上，不考虑修改。那么我们可以令 $last_i$ 表示点 i 的祖先中最近的不是 1 的节点是哪个。然后从 l, r 分别往上跳到 LCA，每次由点 x 跳到 $last_x$ ，相邻两个数之间插入最多 24 个 1。令 $M = 24$ ，那么总共我们可以得到一个长度 $O(M^2)$ 的有用的序列，对这个序列进行 dp 即可。复杂度 $O(nM^3)$ 。

E - Echoes of 24

- 现在考虑带修。那么每个节点的 $last_i$ 可能发生变化，我们考虑用树链剖分维护这一过程。线段树维护某个区间内是否存在一个 $a_i > 1$ ，那么在线段树上二分即可求出最近的不是 1 的节点编号。
- 下面分析该过程的时间复杂度。我们最多跳 $M/2$ 次 $last_x$ ，但是树链剖分之后从一个点到根节点最多只会经过 $\log n$ 条重链。所以总时间复杂度为 $O(n(M + \log n) \log n)$ 。

E - Echoes of 24

- 现在瓶颈在于 dp。但是我们可以注意到，有 $O(M^2)$ 个数为 1，只有 $O(M)$ 个数大于 1。
- 我们可以考虑使用位运算优化。具体的，我们令 dp 为当前的 dp 值。如果 dp 的第 i 位为 1，那么就可以达到 i。当我们要处理 1 的时候，我们可以直接让 $dp = dp | (dp \ll 1)$ 再去掉 $1 \ll 25$ （如果有），否则正常 dp。
- 下面分析该过程的时间复杂度。有 $O(M^2)$ 个 1，单次处理时间复杂度 $O(1)$ ；有 $O(M)$ 个非 1 的数，单次处理时间复杂度 $O(M)$ 。总时间复杂度 $O(M^2)$ 。

E - Echoes of 24

- 综上，该题可以做到总时间复杂度 $O(n \log n + q(M + \log n) \log n)$ ，牛客评测机 1500ms。
- 但是，还有优化！

E - Echoes of 24

- 有另一个结论：当路径长度大于 24 时，我们只需要看其中大于 2 的数的和是否超过 24 即可。如果不超过那么就可以，否则不可以。
- 证明其实很显然。当和大于 24 时，怎么救都救不回来；和小于等于 24 时。可以用 1 去补上。

E - Echoes of 24

- 现在我们相当于单点修改然后询问路径上所有数的和。这可以经典的单 log 完成。
- 我们考虑维护一个点到根的路径上所有点的值，那么单点修改的时候只有它的子树会受到影响，实质上就是一个在 dfs 序上的区间加，单点求和然后差分一下。
- 然后当路径长度 ≤ 24 的时候直接暴力 dp，利用 bitset 可以优化。牛客实测约 900ms。



E - Echoes of 24

- 最后补充一下，空间限制 256MB 是为了卡掉一些 $nM^2 \log n$ 的做法，正常做法可以轻松通过。



A - All kinds of dice

题目大意

- 给一个竞赛图，构造 N 个骰子使得骰子间的胜负关系与竞赛图相符。
- $N \leq 100$ ，构造的骰子总面数不超过 10^5 。

A - All kinds of dice

题解

- 考虑如下 N 个两面骰子：第 i 个骰子包含数字 i 和 $2N + 1 - i$ （其中 $1 \leq i \leq N$ ）。此时任意两个骰子相互之间的胜率都是 $1/2$ ；
- 为了制造不等关系，我们可以交换第 i 个和第 $i + 1$ 个骰子的较小数字。此时我们便制造出了一对不等关系；
- 示例：考虑 N 个两面骰子 $(1, 2N), (2, 2N - 1), \dots, (i - 1, 2N - i + 2), (i, 2N - i), (i + 1, 2N - i + 1), (i + 1, 2N - i), \dots, (N, N + 1)$ ，此时骰子 $i + 1$ 胜过骰子 i ，且其他任意两个骰子相互之间的胜率均为 $1/2$ 。

A - All kinds of dice

题解

- 更一般的，对于任意的排列 P ，我们让第 P_i 个骰子包含数字 i 和 $2N + 1 - i$ ，交换 P_i 与 P_{i+1} 号骰子的较小（或较大数字），可制造任意一对不等关系；
- 因此，我们可以制造 $M = \Theta(N^2)$ 组 N 个两面骰子，每一组中恰好有一对不等关系。我们可以由此构造满足条件的 N 个具有 $2M$ 面的骰子：将第 k 组的第 i 个骰子两面写有的数字加上 $2Nk$ 后加入到新的第 i 号骰子中。
- 最后，利用完全图的匹配划分，一个骰子可以制造出 $\Theta(N)$ 对不等关系，这样可以将骰子个数 M 减少至 $M = \Theta(N)$ 从而满足条件。



K - Kindred Sums

题目大意

- 给定 $N = 1000$ 个正整数以及模数 $M = 10^{18}$ ，找两个在模 M 意义下子集和一致的子集。

K - Kindred Sums

题解

- 可能有一些比较有道理的乱搞能过。我们这里分享一个有正确性保障的做法。
- 问题等价于给每个数赋权重 $c \in \{-1, 0, 1\}$ ，带权重和为 M 的倍数。
- 令 $B = 31$ 。我们将 B^2 个数平均分为 B 组。在每一组中，利用生日悖论在 $O(M^{1/4})$ 的期望时间找到两个互不相同的子集，其子集和模 $\sqrt{M}$ 一致。这样的话，我们可以给每一组的每个数赋权，使得权重和为 $\sqrt{M}$ 的倍数。此时，每一组可以当成一个元素，元素大小为该组权重和除以 M ，再次使用生日悖论即可得到整体的赋权满足条件。

C - Computational Geometry Problem

题目大意

- 求满足如下条件的长度为 NM 的序列个数（模 998 244 353）。
 - 1 $1 \leq A_i \leq M$ ，且从 1 到 M 的每个数字在 A 中恰好出现 N 次；
 - 2 不存在整数 $1 \leq i < j < k < \ell \leq NM$ 满足 $A_i = A_k$ ， $A_j = A_\ell$ ，且 $A_i \neq A_j$ ；
 - 3 满足 $1 \leq i < NM$ 且 $A_i = A_{i+1}$ 的数量为 K 。
- $N, M \leq 10^7$ 。
- PS：取这个题目的原因是因为出题人在一个计算几何报告上想到了这个问题。

C - Computational Geometry Problem

题解

- 我们只提供本题的思路，具体计算留给大家练习。
- 第一步：我们考虑将满足条件序列映射到带标号有根树 (T, r) 。对于符合条件的序列，我们将相同连续字符压缩成单一字符得到压缩序列 B 。我们将 B_1 作为分隔符，将序列 B 分割成若干段，递归构造带标号有根树 (T, r) 的根为 B_1 ，子树为非空段递归执行此过程得到的带标号有根树。不存在 $xyxy$ 形的子序列保证了子树的标号不重合（因此是一颗带标号有根树）。

C - Computational Geometry Problem

题解

- 对于每一个带标号有根树 (T, r) ，我们可以计算出有多少满足条件的序列映射到此有根树：仅取决于度数序列以及根的度数。
- 第二步：考虑有根树与 Prufer 序列的一一对应关系，利用 Prufer 序列中数值的出现次数与度数序列的关系，将问题转化为 $[n]^{n-2}$ 序列的带权求和问题进行计算。
- 最终答案： $(M-1)! \binom{NM-M}{M-1-r} \binom{M}{r}$ ，其中 $r = 2(M-1) + K - (NM-1)$ 。

M - Monopoly

题目大意

- 给定一个非负整数序列，每次可以选择若干个数，将这些数替换为平均数下取整。问若干次操作后序列最大值最小可能是多少。
- $N, a_i \leq 50$ 。

M - Monopoly

题解

- 不妨对序列排序。我们有如下两个观察：
 - 1 $[0, 1, 3, 7, \dots, 2^k - 1]$ 可以归零。
 - 2 在序列末尾添加一个大于或等于当前序列最大值的数字，答案不降。
- 基于这两点观察，我们只需要考虑前 $\lceil \log_2 A_{\max} \rceil = 6$ 个数。此时状态空间规模已经可以支持搜索。

J - Journey Around the World

题目大意

- 给定一个网格图，左右两边也连通，上下不连通，所有边都是无向边并且权值为 1 或 2。问从第一行每个点到最后一行同列的点经过每列至少一次的最短路长度。
- $3 \leq n \leq 200$ 。

J - Journey Around the World

题解

- 容易发现最优解从起点到终点总可以转不超过一圈。
 - 1 如果最优解转了一圈，从起始点到终点转了奇数圈的最短路就是答案。
 - 2 如果最优解只能转零圈。不妨设最优解先到达极左点再到达极右点。如果极右点在极左点至少右边 n 列，在第一个极左点处笔直向下直到接上最后一个极右点到终点的路径会得到一个转了一圈的最优解，因此极右点在极左点右边 $n - 1$ 列。从第一个极左点的地方笔直向下再向左一格到达最后一个极右点的位置转了一圈不是最优解，所以第一个极左点和最后一个极右点的行差至少是 $n - 2$ 并且中间的路径只能向右或者向下。
- 时间复杂度 $O(n^3)$ 。

H - Highs and Lows

题目大意

- 求一个次数最小的多项式 f 满足 $f(i) \equiv a_i \pm 1 \pmod M$ 。
- $n \leq 30$, $M \leq 10^9$ 。

H - Highs and Lows

题解

- 首先考虑 M 为质数的情形，判定是否存在一个 k 次多项式满足条件。
- 给定 $f(0), f(1), \dots, f(N-1)$ 的值，一定存在一个 $N-1$ 次多项式 $f = \sum_{k=0}^{N-1} c_k x^k$ 满足条件，且系数为函数值的线性组合，即 $[c_0, c_1, \dots, c_{N-1}]^T = A \cdot [f(0), f(1), \dots, f(N-1)]^T$ 。
- 因此，问题等价于是否存在一组赋值 $\epsilon \in \{-1, +1\}^N$ 满足

$$A' \cdot [f(0) + \epsilon(0), f(1) + \epsilon(1), \dots, f(N-1) + \epsilon(N-1)]^T = [0, 0, \dots, 0]^T,$$

其中 A' 为 A 的后 $N-1-k$ 行，这一问题利用 meet in the middle 以及 hash 可在 $\text{poly}(n) \cdot 2^{n/2}$ 的时间内解决。

H - Highs and Lows

题解

- 当 M 非质数时，我们考虑下降幂多项式 $f = \sum_{i=0}^{N-1} c_i x^i$ 。此时，给定 $f(0), f(1), \dots, f(N-1)$ 的值，我们可以有如下方程组

$$\sum_{i=0}^k \binom{k}{i} \cdot i! c_i = \sum_{i=0}^k c_i k^i \equiv f(k) \pmod{M},$$

因此， $i!c_i$ 可写为 $f(0), f(1), \dots, f(i)$ 的线性组合。同样利用 meet in the middle 以及 hash 可在 $\text{poly}(n) \cdot 2^{n/2}$ 的时间内解决问题。

THANKS!

AC.NOWCODER.COM